**UNIVERSIDAD POLITÉCNICA DE PACHUCA**




PROGRAMA EDUCATIVO TECNOLOGÍAS DE LA INFORMACIÓN E INOVACIÓN DIGITAL

# DESARROLLO E IMPLEMENTACIÓN DE FILAS Y LAYOUTS HORIZONTALES EN JETPACK COMPOSE

Práctica 3: Contenedores Row y LazyRow

**Curso:** Aplicaciones Móviles / Desarrollo Móvil
**Autor:** Cervantes Gonzalez Johan Baru
**Profesor:** Montoya Cerro Miguel Angel
**Lugar y Fecha:** Zempoala, Hidalgo, 09 de Octubre de 2026

# Índice


**1. Introducción** **3**
- 1.1. Contextualización . . . 3
- 1.2. Planteamiento del Problema . . . 3
- 1.3. Objetivos . . . 3
  - 1.3.1. Objetivo General . . . 3
  - 1.3.2. Objetivos Específicos . . . 3
- 1.4. Justificación . . . 3

**2. Marco Teórico** **4**
- 2.1. Definición de Conceptos Clave . . . 4

**3. Desarrollo** **4**
- 3.1. Análisis Comparativo . . . 4
- 3.2. Guía Paso a Paso . . . 4
  - 3.2.1. Paso 1: Configuración Inicial del Proyecto . . . 4
  - 3.2.2. Paso 2: Definición de la Función Principal y Estructura Row . . . 5
  - 3.2.3. Paso 3: Verificación de Alineación Inicial . . . 6
  - 3.2.4. Paso 4: Intento de Distribución y Limitación de Tamaño . . . 7
  - 3.2.5. Paso 5: Expansión de Ancho con Modificadores . . . 7
  - 3.2.6. Paso 6: Distribución Espaciada (SpaceBetween) . . . 8
  - 3.2.7. Paso 7: Ajuste de Márgenes e Internos (Padding) . . . 8
  - 3.2.8. Paso 8: Identificación del Desbordamiento por Exceso de Contenido 9
  - 3.2.9. Paso 9: Migración a LazyRow . . . 9
  - 3.2.10. Paso 10: Encapsulación en Bloques Item . . . 9
  - 3.2.11. Paso 11: Verificación de Desplazamiento Horizontal . . . 10
  - 3.2.12. Paso 12: Comprobación de Doble Desplazamiento en Rotación . 10

**4. Conclusiones** **10**

**5. Anexos** **11**
- 5.1. Anexo A: Recursos de Apoyo . . . 11

**6. Referencias** **11**

# Índice de figuras

1. Creación del proyecto base en Android Studio . . . . . 5
2. Definición inicial de la función y el composable Row . . . . . 6
3. Elementos de texto organizados horizontalmente sin formato . . . . . 6
4. Visualización de la fila en la vista previa . . . . . 7
5. Intento de centrar sin el modificador de ancho completo . . . . . 7
6. Aplicación del modificador fillMaxWidth() . . . . . 8
7. Aplicación de Arrangement.SpaceBetween . . . . . 8
8. Inclusión de padding(16.dp) . . . . . 9
9. Adición de múltiples textos provocando desbordamiento en código . . . 9
10. Resultado final de la interfaz construida . . . . . 10

# Índice de tablas

1. Comparativa entre Row/LazyRow en Compose y LinearLayout horizontal 4

# 1. Introducción

## 1.1. Contextualización

En el desarrollo de interfaces modernas para Android mediante el paradigma declarativo de Jetpack Compose, la maquetación en una sola dimensión resulta insuficiente para crear experiencias de usuario completas. Tras comprender el uso de las columnas verticales, se vuelve indispensable dominar la distribución horizontal de elementos visuales.

## 1.2. Planteamiento del Problema

Al estructurar componentes contiguos de manera horizontal sin un contenedor apropiado, los elementos se superponen o colapsan en el origen del lienzo. Asimismo, el manejo estático de componentes horizontales presenta problemas de desbordamiento de pantalla cuando el contenido excede el ancho del dispositivo o ante cambios de orientación.

## 1.3. Objetivos

### 1.3.1. Objetivo General

Implementar interfaces de usuario orientadas horizontalmente mediante los contenedores `Row` y `LazyRow` en Jetpack Compose, aplicando modificadores de dimensión, alineación y desplazamiento horizontal dinámico.

### 1.3.2. Objetivos Específicos

- Implementar el componente `Row` para la disposición secuencial de elementos de izquierda a derecha.
- Evaluar el uso de modificadores de espacio como `fillMaxWidth()`, `padding()` y propiedades de distribución (`Arrangement.SpaceBetween`).
- Diagnosticar las limitaciones de capacidad y recorte visual al incorporar múltiples elementos en un `Row` estándar.
- Resolver el desbordamiento de pantalla aplicando la estructura perezosa `LazyRow` e integrando bloques `item`.

## 1.4. Justificación

Esta práctica permite dominar la bidimensionalidad en la maquetación de Compose, combinando ejes verticales y horizontales para construir vistas flexibles que se adapten a diferentes resoluciones de pantalla y orientaciones del dispositivo.

# 2. Marco Teórico

## 2.1. Definición de Conceptos Clave

**Row:** Contenedor fundamental en Jetpack Compose que organiza a sus elementos componentes de forma horizontal secuencial (de izquierda a derecha). Equivale al `LinearLayout` con orientación horizontal del modelo XML tradicional.

**HorizontalArrangement:** Propiedad de configuración interna que define la distribución de los elementos a lo largo del eje horizontal dentro de una fila (ej. `Center`, `SpaceBetween`, `SpaceAround`).

**LazyRow:** Componente de renderizado optimizado para desplazamiento horizontal. Compone y dibuja únicamente las vistas visibles en pantalla, equivalente a un `RecyclerView` de orientación horizontal.

**Item Scope:** Entorno DSL utilizado dentro de las listas perezosas (`LazyRow` / `LazyColumn`) para envolver composables individuales.

# 3. Desarrollo

## 3.1. Análisis Comparativo

Tabla 1: Comparativa entre Row/LazyRow en Compose y LinearLayout horizontal

| **Criterio** | **Layout Clásico (XML)** | **Jetpack Compose** |
|---|---|---|
| Contenedor Básico | `LinearLayout` (horizontal) | `Row` |
| Desplazamiento | `HorizontalScrollView` / `RecyclerView` | `LazyRow` |
| Distribución de Espacio | Atributos `layout_weight` | Modificador `weight()` o `Arrangement` |

## 3.2. Guía Paso a Paso

### 3.2.1. Paso 1: Configuración Inicial del Proyecto

Se configuró el entorno de trabajo reutilizando el proyecto base de desarrollo Android en Android Studio.

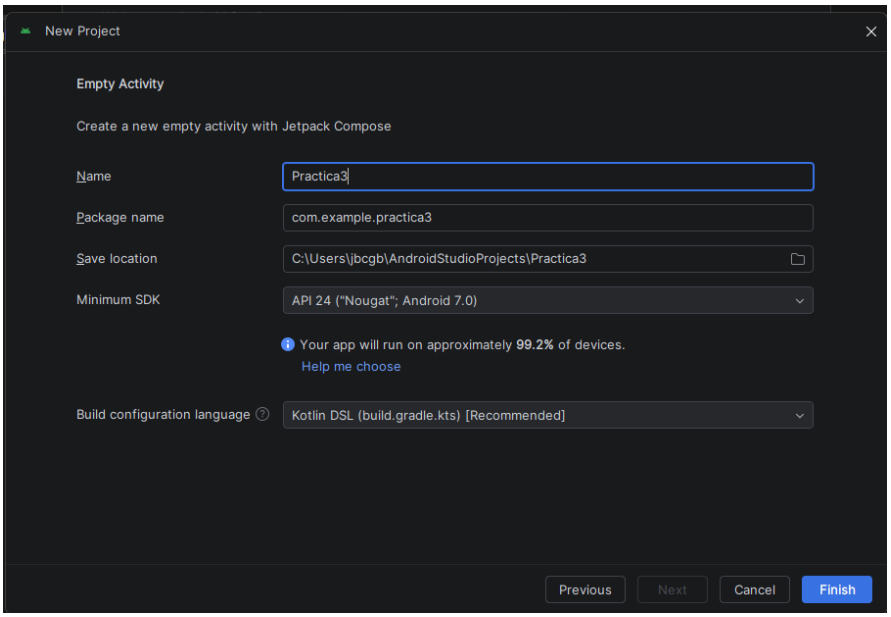



Figura 1: Creación del proyecto base en Android Studio

### 3.2.2. Paso 2: Definición de la Función Principal y Estructura Row

Se definió el bloque `Row` para albergar elementos de texto horizontales.

```
@Composable
fun App() {
    Row {
        Text(text = "Java")
        Text(text = "Kotlin")
    }
}
```



```
1 Usage
@Composable
fun FilaBasica() {
    Row {
        Text(text = "Java ")
        Text(text = "Kotlin ")
        Text(text = "Android")
    }
}

@Preview(showBackground = true)
@Composable
fun PreviewFilaBasica() {
    FilaBasica()
}
```

Figura 2: Definición inicial de la función y el composable Row

### 3.2.3. Paso 3: Verificación de Alineación Inicial

Al agregar múltiples componentes de texto dentro de la fila, se observa su posicionamiento secuencial pegado a la izquierda.



```
@Composable
fun Paso3TextosJuntos() {
    Row {
        Text(text = "Java ")
        Text(text = "Kotlin ")
        Text(text = "Android ")
    }
}
```

Figura 3: Elementos de texto organizados horizontalmente sin formato

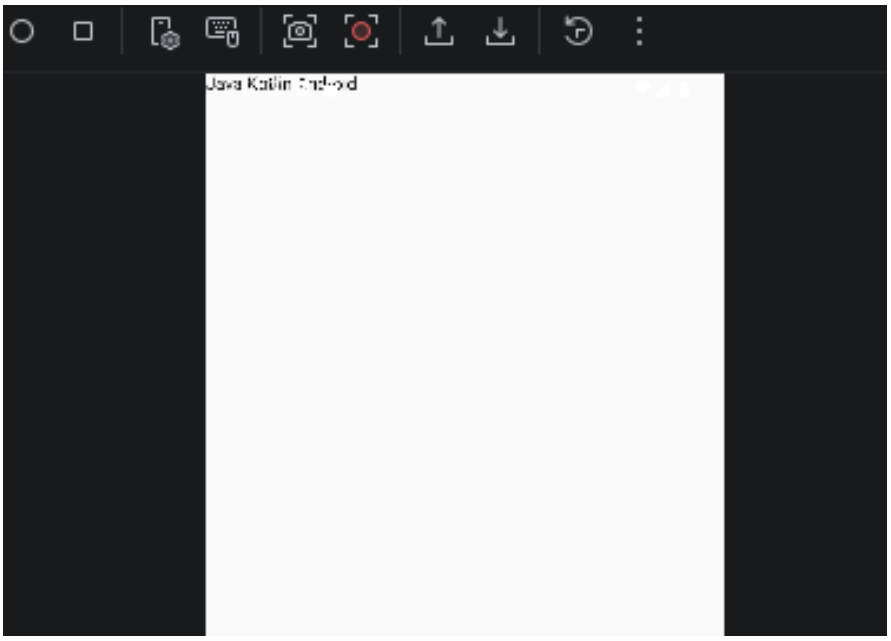


Figura 4: Visualización de la fila en la vista previa

### 3.2.4. Paso 4: Intento de Distribución y Limitación de Tamaño

Se intentó aplicar el centrado mediante `horizontalArrangement = Arrangement.Center`, pero no surtió efecto debido a que la fila por defecto adopta el tamaño envolvente de sus componentes (`wrap_content`).

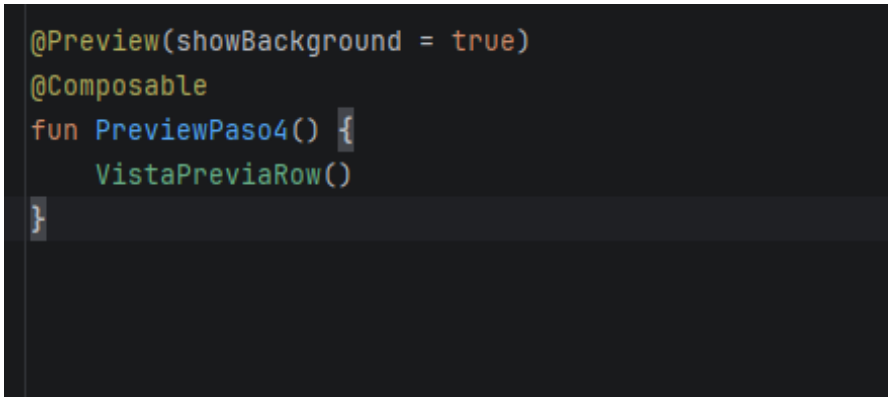



Figura 5: Intento de centrar sin el modificador de ancho completo

### 3.2.5. Paso 5: Expansión de Ancho con Modificadores

Se aplicó el modificador `Modifier.fillMaxWidth()` para forzar al contenedor a expandirse horizontalmente en toda la pantalla.

```
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.Center
```

```
) {
    Text(text = "Java")
    Text(text = "Kotlin")
}
```

```
@Composable
fun Paso5IntentoCentrado() {
    Row(
        horizontalArrangement = Arrangement.Center
    ) {
        Text(text = "Java ")
        Text(text = "Kotlin ")
    }
}
```

Figura 6: Aplicación del modificador fillMaxWidth()

### 3.2.6. Paso 6: Distribución Espaciada (SpaceBetween)

Se cambió la propiedad de arreglo a `Arrangement.SpaceBetween` para distribuir los elementos de extremo a extremo.

```
@Composable
fun PasoóRowCentradoCompleto() {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.Center
    ) {
        Text(text = "Java ")
        Text(text = "Kotlin ")
    }
}
```

Figura 7: Aplicación de Arrangement.SpaceBetween

### 3.2.7. Paso 7: Ajuste de Márgenes e Internos (Padding)

Se asignó un `padding(16.dp)` para evitar que los componentes queden pegados a los bordes de la pantalla.

```kotlin
@Composable
fun Paso7SpaceBetween() {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text(text = "Java")
        Text(text = "Kotlin")
        Text(text = "Android")
    }
}
```

Figura 8: Inclusión de padding(16.dp)

### 3.2.8. Paso 8: Identificación del Desbordamiento por Exceso de Contenido

Se ingresaron múltiples elementos de texto repetidos ("Suscríbete") superando el ancho de la pantalla física.

```kotlin
@Composable
fun Paso8ConPadding() {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding( all = 16.dp),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text(text = "Java")
        Text(text = "Kotlin")
        Text(text = "Android")
    }
}
```

Figura 9: Adición de múltiples textos provocando desbordamiento en código

### 3.2.9. Paso 9: Migración a LazyRow

Se reemplazó el componente `Row` por un `LazyRow` para habilitar el soporte de lista horizontal perezosa.

### 3.2.10. Paso 10: Encapsulación en Bloques Item

Se integró el bloque DSL `item { ... }` requerido obligatoriamente por los contenedores perezosos.

```
LazyRow(
    modifier = Modifier.fillMaxWidth()
) {
    item {
        Text(text = "Java")
        Text(text = "Kotlin")
        Text(text = "Suscr bete")
    }
}
```

### 3.2.11. Paso 11: Verificación de Desplazamiento Horizontal

Se ejecutó la aplicación comprobando que el desplazamiento lateral se realiza de forma fluida y sin recortes de pantalla.

### 3.2.12. Paso 12: Comprobación de Doble Desplazamiento en Rotación

Al girar el dispositivo a orientación horizontal, se validó la coexistencia del desplazamiento vertical (`LazyColumn` padre) y desplazamiento horizontal (`LazyRow` interno).

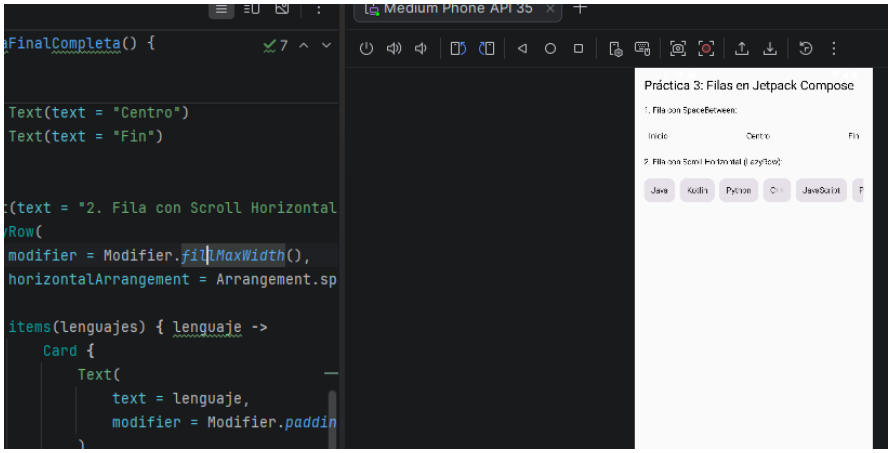



Figura 10: Resultado final de la interfaz construida

# 4. Conclusiones

- El uso de `Row` permite una maquetación clara y estructurada en el eje horizontal, complementando el diseño vertical de las aplicaciones.
- Los contenedores de fila requieren explícitamente modificadores como `fillMaxWidth()` para distribuir adecuadamente los elementos a través del espacio libre.
- `LazyRow` resulta fundamental para evitar recortes de interfaz cuando el número de elementos horizontales es dinámico o sobrepasa la resolución del dispositivo.

# 5. Anexos

## 5.1. Anexo A: Recursos de Apoyo

- Documentación en git: https://github.com/Elprojobra/Aplicaciones-moviles
- Guía de listas perezosas: https://developer.android.com/jetpack/compose/lists

# 6. Referencias

Android Developers. (2024). *Layouts in Jetpack Compose*. Google. https://developer.android.com/jetpack/compose/layouts

Android Developers. (2024). *Lists and Performance in Compose*. Google. https://developer.android.com/jetpack/compose/lists